CHAPTER 1

INTRODUCTION

Power transformers are among the most critical components of electrical power distribution systems. Their reliable operation is essential for ensuring uninterrupted power supply to industrial, commercial, and residential consumers. Unexpected transformer failures can result in power outages, equipment damage, increased maintenance costs, and reduced system reliability. Therefore, continuous monitoring and early fault detection have become important aspects of modern power system maintenance.

Traditional maintenance approaches are generally based on periodic inspections and scheduled servicing. Although these methods help identify certain operational issues, they often fail to detect hidden faults at an early stage. As a result, maintenance activities may either be performed unnecessarily or delayed until significant equipment degradation has occurred. To overcome these limitations, predictive maintenance techniques based on Machine Learning (ML) have emerged as an effective solution for transformer condition monitoring.

Dissolved Gas Analysis (DGA) is one of the most widely used diagnostic techniques for assessing transformer health. During transformer operation, electrical and thermal faults generate characteristic gases such as Hydrogen (H_2), Carbon Monoxide (CO), Ethylene (C_2H_4), and Acetylene (C_2H_2). By analyzing the concentration and trends of these gases, it is possible to identify potential faults and estimate the condition of the transformer.

This project proposes a Machine Learning-based Transformer Fault Detection and Remaining Useful Life (RUL) Prediction System. The system utilizes DGA data and advanced machine learning algorithms including LightGBM and XGBoost to classify transformer fault conditions and estimate equipment lifetime. The developed framework can identify Normal, Partial Discharge, Thermal Fault, and Electrical Fault conditions while providing confidence scores and health assessments.

To improve transparency and trust in the prediction process, Explainable Artificial Intelligence (XAI) techniques based on SHAP analysis are incorporated. A Django-based web application provides an interactive platform for dataset management, fault prediction, health monitoring, visualization, and report generation. The proposed system enables intelligent transformer monitoring, supports predictive maintenance planning, and contributes to improved reliability and operational efficiency in power distribution systems.

CHAPTER 2

LITERATURE SURVEY

2.1) Predictive Maintenance for Distribution System Operators in Increasing Transformers' Reliability

Author: V. Vita (2023)

This paper presents predictive maintenance techniques for improving transformer reliability in power distribution systems. The study focuses on monitoring transformer conditions, detecting faults at an early stage, and reducing unexpected failures. The results show that predictive maintenance can significantly improve operational efficiency and reduce maintenance costs.

2.2) Machine Learning based Predictive Maintenance in Distribution Transformers

Author: V. Biradar and D. Kakeri (2024)

The authors proposed a machine learning-based framework for predictive maintenance of distribution transformers. Various machine learning algorithms were used to analyze transformer data and predict potential failures. The study demonstrated that machine learning models can effectively identify fault patterns and support maintenance decision-making.

2.3) Deep Learning for Transformer Fault Diagnosis Using Dissolved Gas Analysis

Author: M. Khalid, Z. Bin, and A. Khan (2023)

This research utilized deep learning techniques for transformer fault diagnosis using Dissolved Gas Analysis (DGA) data. The model analyzed concentrations of gases generated within transformers and accurately classified fault conditions. The study highlighted the effectiveness of deep learning in improving diagnostic accuracy and transformer health monitoring.

2.4) Transformer Health Index Prediction Using Random Forest

Author: **R. Sharma and P. K. Jain (2023)**

This paper proposed a Random Forest-based model for predicting transformer health indices. Various transformer operating parameters were analyzed to estimate the health condition of equipment. The proposed approach improved maintenance planning by providing reliable health assessments and supporting predictive maintenance strategies.

2.5) Condition Monitoring of Transformers Using Explainable AI

Author: J. He et al. (2024)

The authors introduced Explainable Artificial Intelligence (XAI) techniques for transformer condition monitoring. The study combined machine learning models with interpretability methods to explain prediction outcomes and identify influential factors affecting transformer health. The approach increased transparency and trust in AI-based maintenance systems.

CHAPTER 3

SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

Current transformer maintenance systems mainly depend on periodic inspections and manual analysis of Dissolved Gas Analysis (DGA) reports. Traditional maintenance approaches often detect faults only after significant degradation has occurred.

Most existing systems focus on monitoring transformer parameters but do not provide intelligent fault prediction, Remaining Useful Life estimation, or explainable decision-making mechanisms. Real-time web-based predictive maintenance platforms are also limited.

Disadvantages of Existing System

- Manual analysis of transformer condition
- Limited fault prediction capability
- No Remaining Useful Life estimation
- Lack of automated health assessment
- Difficult interpretation of transformer health status
- No explainable AI support
- Limited visualization and reporting features
- No centralized web-based monitoring platform
- Increased maintenance cost due to late fault detection
- Reduced equipment reliability and availability

3.2 Proposed System

The proposed system introduces an intelligent Transformer Fault Detection and RUL Prediction Platform using Machine Learning and Explainable AI.

Data Acquisition

Transformer Dissolved Gas Analysis (DGA) data is collected through CSV files containing:

- H_2 (Hydrogen)
- CO (Carbon Monoxide)
- C_2H_4 (Ethylene)
- C_2H_2 (Acetylene)

Feature Engineering

Statistical and trend-based features are extracted from gas measurements including:

- Mean
- Standard Deviation
- Minimum
- Maximum
- Range
- Median
- Quartiles
- Skewness
- Kurtosis
- Slope Analysis
- Gas Ratios

Fault Classification

Machine Learning models classify transformer conditions into:

- Normal
- Partial Discharge

- Thermal Fault
- Electrical Fault

Remaining Useful Life Prediction

The system predicts the Remaining Useful Life (RUL) of transformers using ensemble machine learning models.

The platform computes:

- Health Score
- Health Status
- Risk Level
- Maintenance Recommendation

Explainable AI Analysis

SHAP-based Explainable AI identifies:

- Primary Fault Driver
- Secondary Driver
- Tertiary Driver
- Most Influential Features

Dashboard Analytics

Interactive dashboards provide:

- Dataset Analysis
- Fault Distribution
- Model Performance Comparison
- RUL Analysis
- Health Monitoring
- Gas Trend Visualization
- Prediction Reports

Web Deployment

The Django platform supports:

- User Registration and Login
- Admin Management
- Dataset Upload
- Prediction System
- Report Generation
- Historical Monitoring
- Dashboard Visualization
-

Advantages of Proposed System

- Automatic transformer fault detection
- Accurate Remaining Useful Life prediction
- Real-time health monitoring
- Explainable AI-based decision support
- Early fault identification
- Reduced maintenance costs
- Improved transformer reliability
- Interactive analytical dashboards

3.3 System Specification

Hardware Requirements

- System : Intel Core i5.
- Hard Disk : 1TB.
- Monitor : 15'' LED
- Input Devices : Keyboard, Mouse
- Ram : 16GB.

Software Requirements

- Operating system : Windows 10.
- Coding Language : Python
- Tool : PyCharm, Visual Studio Code
- Database : SQLite

CHAPTER 4

SYSTEM STUDY AND DESIGN

4.1 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

Economic Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

4.2 System Architecture

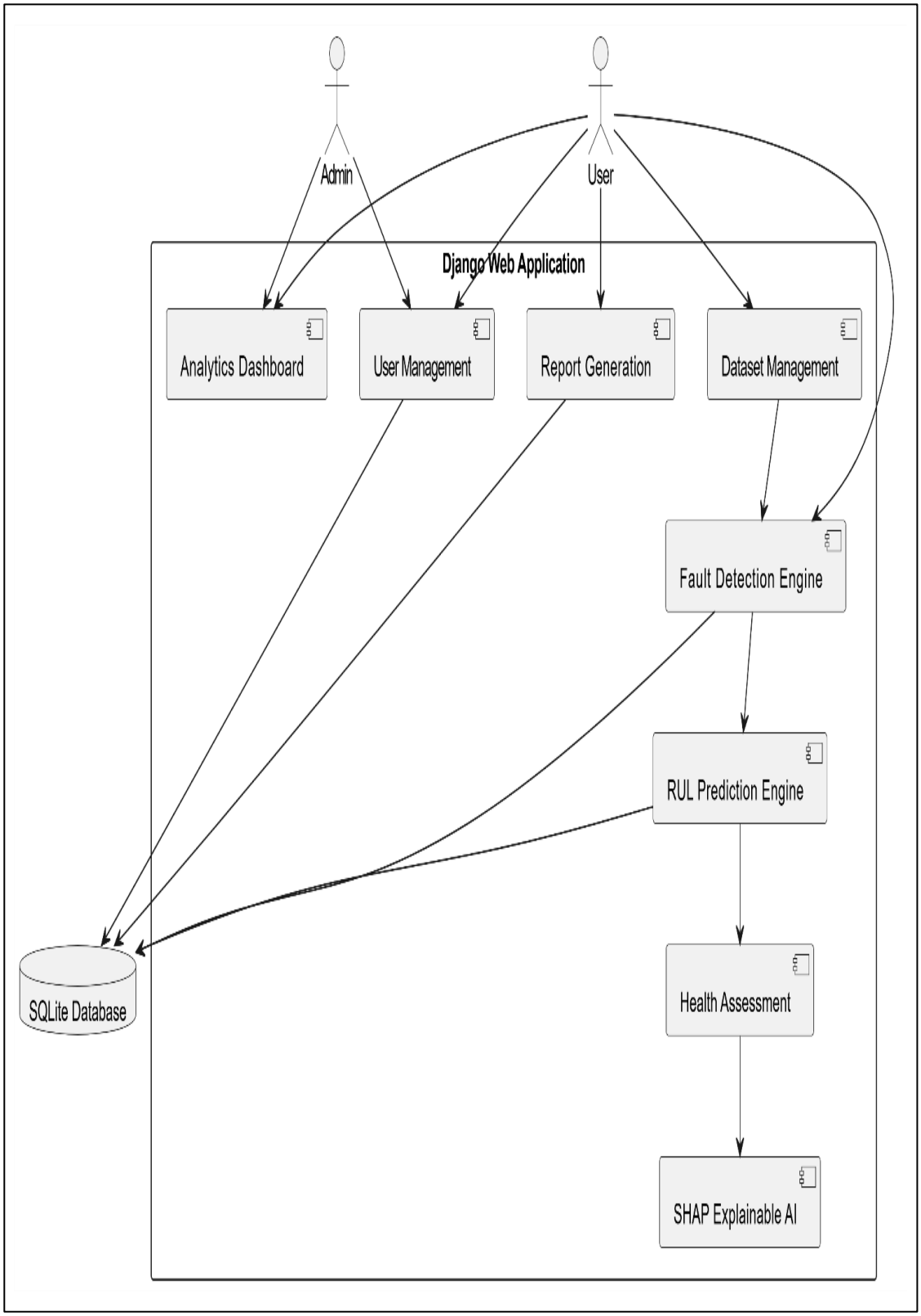


Figure 4.1 System Architecture

4.2.1 Data Flow Diagram:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

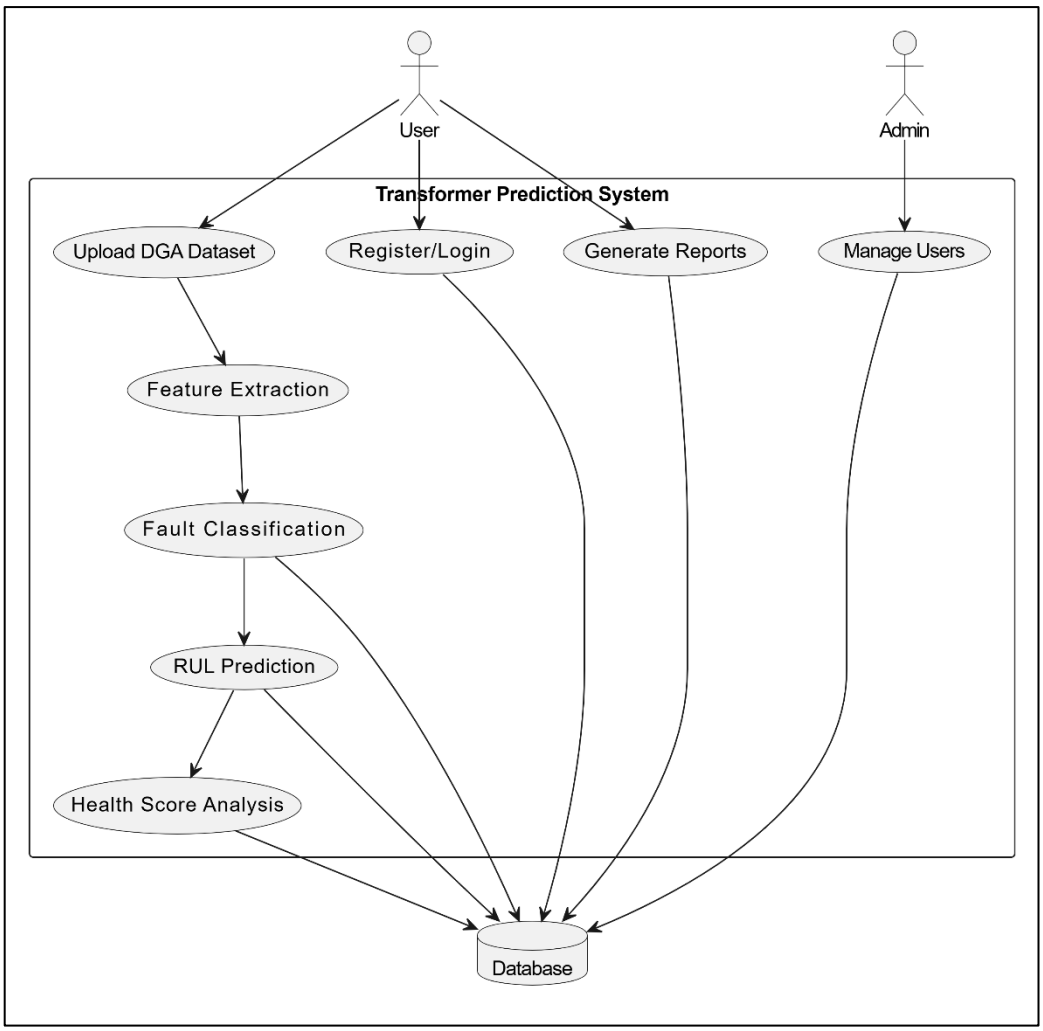


Figure 4.2 Data Flow Diagram

4.2.2 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

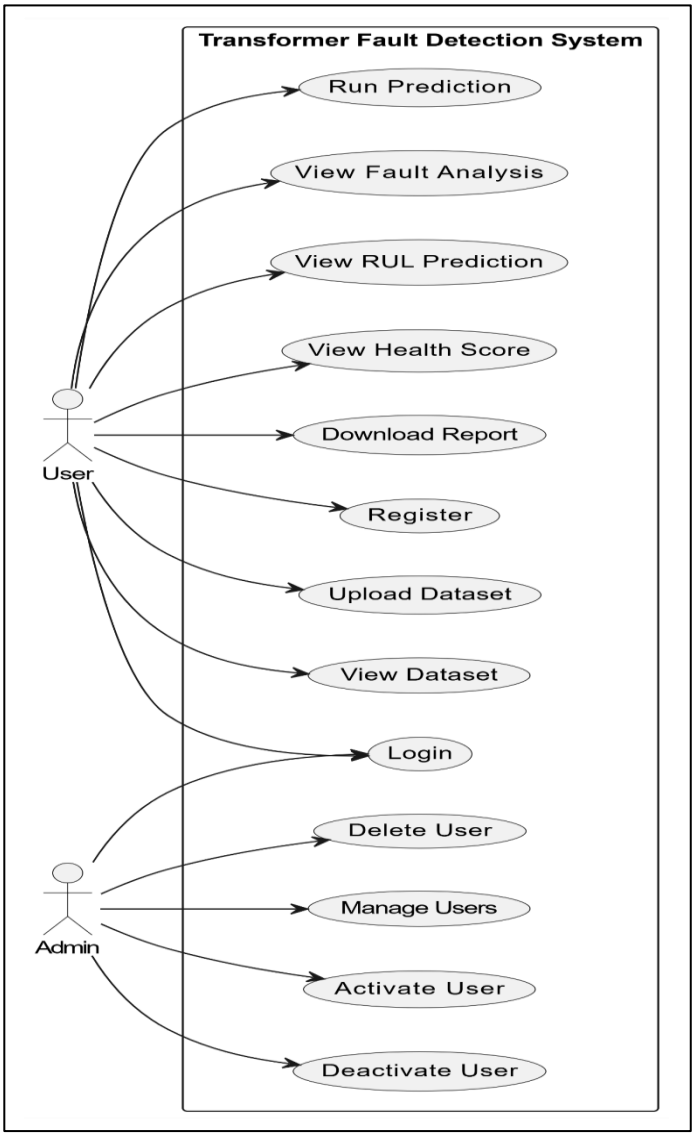


Figure 4.3 Use Case Diagram

4.2.3 Class Diagram:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

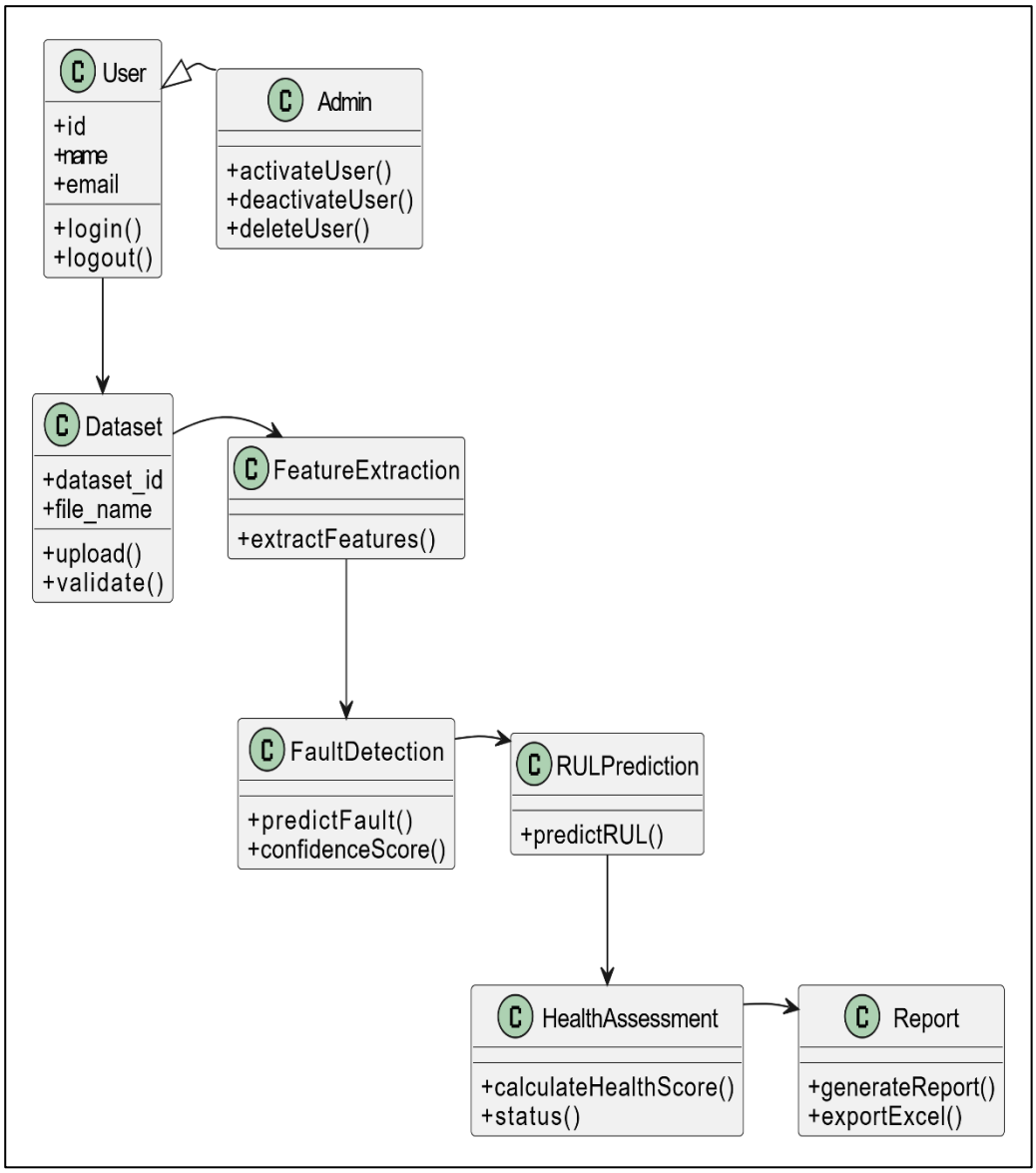


Figure 4.4 Class Diagram

4.2.4 Sequence Diagram:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

.

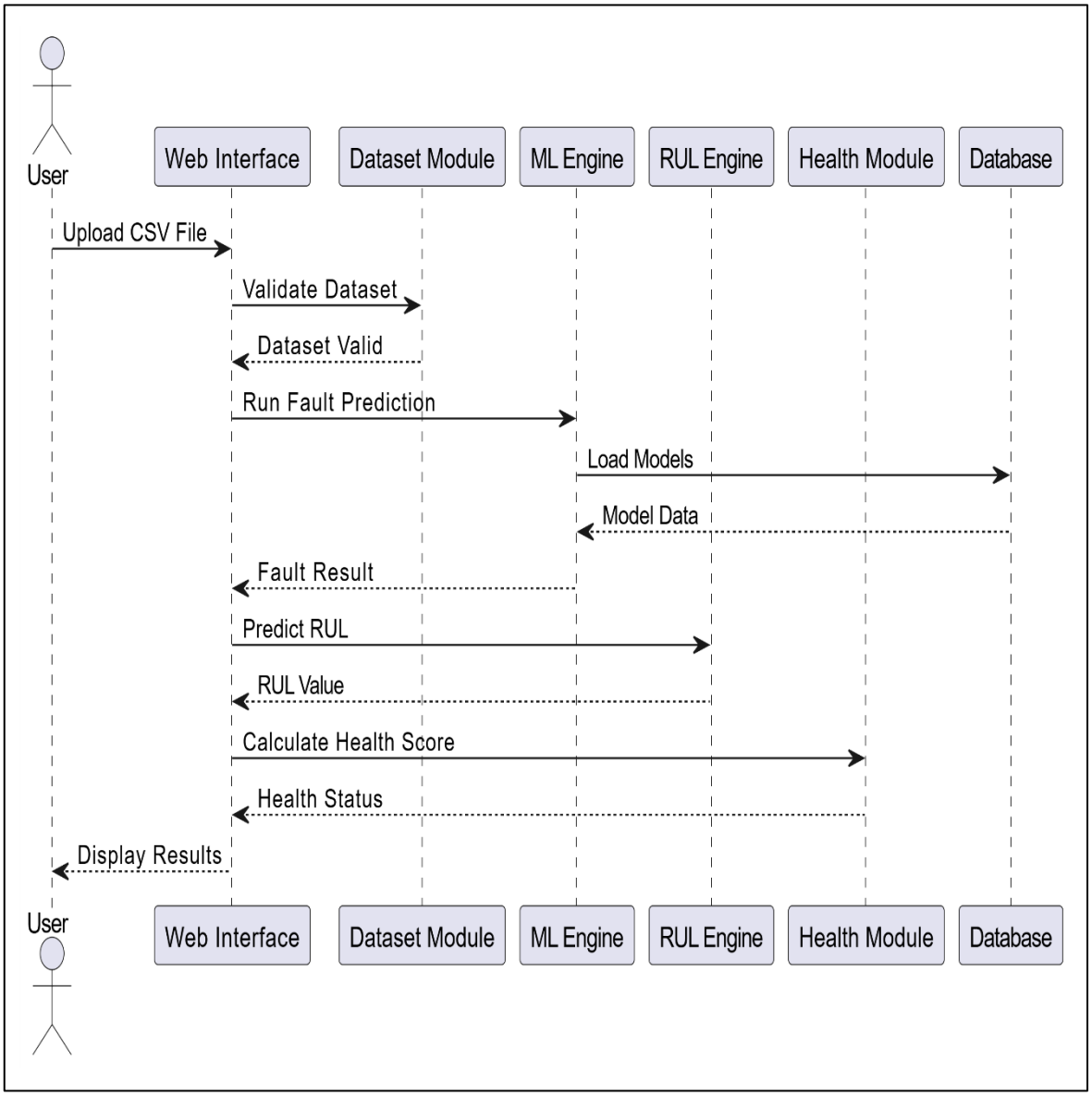


Figure 4.5 Sequence Diagram

4.2.5 Activity Diagram:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

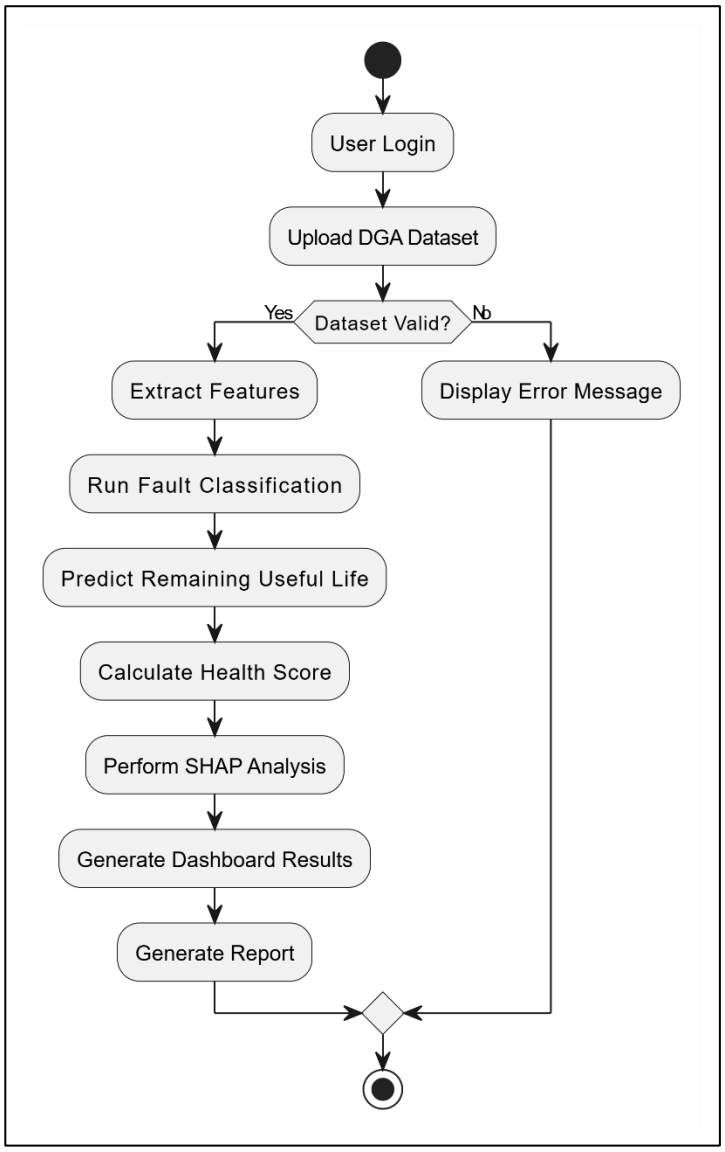


Figure 4.6 Activity Diagram

CHAPTER 5

INPUT AND OUTPUT DESIGN

5.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

OBJECTIVES

1.Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3.When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

5.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2.Select methods for presenting information.

3.Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER 6

SOFTWARE ENVIRONMENT

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation.

Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt –

```
$ python
Python 2.4.3 (#1, Nov 11 2010, 13:34:43)
[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Type the following text at the Python prompt and press the Enter −

```
>>> print "Hello, Python!"
```

Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active. Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file −

```
print "Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows −

```
$ python test.py
This produces the following result –
```

```
Hello, Python!
```

Let us try another way to execute a Python script. Here is the modified test.py file −

```
#!/usr/bin/python
print "Hello, Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows −

```
$ chmod +x test.py
# This is to make file executable
$./test.py
This produces the following result –
Hello, Python!
```

Python Identifiers

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9). Python does not allow punctuation characters such as @, $, and % within identifiers. Python is a case sensitive programming language. Thus, Manpower and manpower are two different identifiers in Python.

Reserved Words

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All the Python keywords contain lowercase letters only.

and, exec, not, assert, finally, or, break, for, pass, class, print, from, continue, global, raise, def, if, return, del, import, try, elif, in, while, else, is, with, except, lambda, yield

Lines and Indentation

Python provides no braces to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced. The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount. For example −

```
if True:
print "True"
else:
print "False"
```

However, the following block generates an error −

```
if True:
print "Answer"
print "True"
else:
print "Answer"
print "False"
```

Thus, in Python all the continuous lines indented with same number of spaces would form a block.

Quotation in Python

Python accepts single ('), double (") and triple ('" or """) quotes to denote string literals, as long as the same type of quote starts and ends the string. The triple quotes are used to span the string across multiple lines. For example, all the following are legal −

```
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is
```

made up of multiple lines and sentences."""

Comments in Python

A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

```
#!/usr/bin/python
# First comment
print "Hello, Python!" # second comment
This produces the following result −
Hello, Python!
```

You can type a comment on the same line after a statement or expression − name = "Madisetti" # This is again comment You can comment multiple lines as follows −

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

Following triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:

'''

This is a multiline

comment.

'''

Using Blank Lines

A line containing only whitespace, possibly with a comment, is known as a blank line and Python totally ignores it. In an interactive interpreter session, you must enter an empty physical line to terminate a multiline statement.

Waiting for the User

The following line of the program displays the prompt, the statement saying “Press the enter key to exit”, and waits for the user to take action −

```
#!/usr/bin/python
raw_input("\n\nPress the enter key to exit.")
```

Here, "\n\n" is used to create two new lines before displaying the actual line. Once the user presses the key, the program ends. This is a nice trick to keep a console window open until the user is done with an application.

Multiple Statements on a Single Line

The semicolon (;) allows multiple statements on the single line given that neither statement starts a new code block. Here is a sample snip using the semicolon.

```
import sys; x = 'foo'; sys.stdout.write(x + '\n')
```

Multiple Statement Groups as Suites

A group of individual statements, which make a single code block are called suites in Python. Compound or complex statements, such as if, while, def, and class require a header line and a suite. Header lines begin the statement (with the keyword) and terminate with a colon (:) and are followed by one or more lines which make up the suite. For example −

```
if expression :
suite
elif expression :
suite
else :
suite
```

Command Line Arguments

Many programs can be run to provide you with some basic information about how they should be run. Python enables you to do this with -h −

```
$ python -h
```

usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ... Options and arguments (and corresponding environment variables):

-c cmd : program passed in as string (terminates option list) -d : debug output from parser (also PYTHONDEBUG=x) -E : ignore environment variables (such as PYTHONPATH) -h : print this help message and exit

Python Lists

The list is a most versatile datatype available in Python which can be written as a list of commaseparated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type. Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"]
```

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing −

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value −

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

```
#!/usr/bin/python
```

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
When the above code is executed, it produces the following result –
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation. To explicitly remove an entire dictionary, just use the del statement. Following is a simple example −

```
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

del dict['Name']; # remove entry with key 'Name'

```
dict.clear();
# remove all entries in dict
```

del dict ;

```
# delete entire dictionary
print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys. There are two important points to remember about dictionary keys −

```
(a) More than one entry per key not allowed. Which means no duplicate key is
allowed. When duplicate keys encountered during assignment, the last assignment
wins. For example –
#!/usr/bin/python
dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print "dict['Name']: ", dict['Name']
When the above code is executed, it produces the following result –
dict['Name']: Manni
(b) Keys must be immutable. Which means you can use strings, numbers or tuples
as dictionary keys but something like ['key'] is not allowed. Following is a
simple example –
```

```
#!/usr/bin/python
dict = {['Name']: 'Zara', 'Age': 7}
print "dict['Name']: ", dict['Name']
When the above code is executed, it produces the following result –
Traceback (most recent call last):
File "test.py", line 3, in <module>
dict = {['Name']: 'Zara', 'Age': 7};
TypeError: unhashable type: 'list'
```

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates −

```
#!/usr/bin/python
tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');
# Following action is not valid for tuples
# tup1[0] = 100;
# So let's create a new tuple as follows
tup3 = tup1 + tup2;
print tup3;
When the above code is executed, it produces the following result –
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded. To explicitly remove an entire tuple, just use the del statement. For example −

```
#!/usr/bin/python
tup = ('physics', 'chemistry', 1997, 2000);
print tup;
```

DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It’s free and open source. Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusabilityand "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.

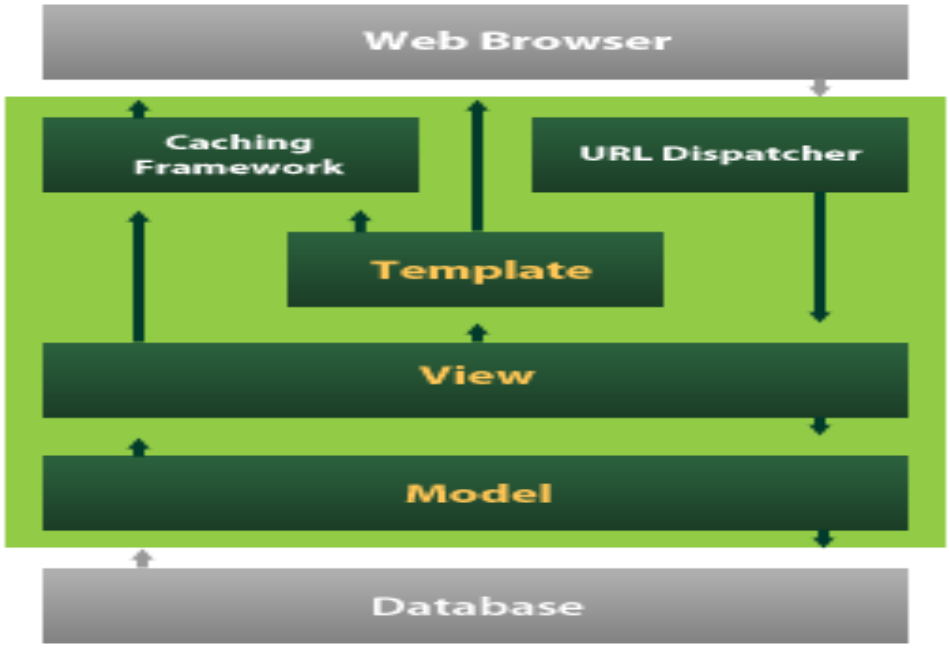


Figure 6.1 Django Flowchart

Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models

Create a Project

Whether you are on Windows or Linux, just get a terminal or a cmd prompt and navigate to the place you want your project to be created, then use this code −

```
$ django-admin startproject myproject
```

This will create a "myproject" folder with the following structure −

```
myproject/
manage.py
myproject/
__init__.py
```

```
settings.py

urls.py

wsgi.py
```

The Project Structure

The “myproject” folder is just your project container, it actually contains two elements −

```
manage.py – This file is kind of your project local django-admin for interacting
with your project via command line (start the development server, sync db...).
To get a full list of command accessible via manage.py you can use the code –

$ python manage.py help
```

The “myproject” subfolder − This folder is the actual python package of your project. It contains four files −

```
__init__.py – Just for python, treat this folder as package.

settings.py – As the name indicates, your project settings.

urls.py – All links of your project and the function to call. A kind of ToC of
your project.

wsgi.py – If you need to deploy your project over WSGI.
```

Setting Up Your Project

Your project is set up in the subfolder myproject/settings.py. Following are some important options you might need to set − DEBUG = True This option lets you set if your project is in debug mode or not. Debug mode lets you get more information about your project's error. Never set it to ‘True’ for a live project. However, this has to be set to ‘True’ if you want the Django light server to serve static files. Do it only in the development mode.

```
DATABASES = {

'default': {

'ENGINE': 'django.db.backends.sqlite3',

'NAME': 'database.sql',

'USER': '',

'PASSWORD': '',

'HOST': '',

'PORT': '',

}

}
```

CHAPTER 7

SYSTEM TESTING AND RESULTS DISCUSSION

7.1 System Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product, and of checking that the software meets its requirements and user expectations. The following types of testing were considered for the system:

- **Unit testing** checks the internal logic of individual units such as forms, the preprocessing functions and the prediction service before they are integrated.
- **Integration testing** checks that the components work together, for example that the trained model saved by the training pipeline is loaded correctly by the prediction view.
- **Functional testing** checks valid and invalid inputs, the functions of each screen, and the outputs produced.
- **System testing** checks the complete integrated application against its requirements.
- **White box testing** uses knowledge of the code structure to test internal paths.
- **Black box testing** tests the application through its inputs and outputs without knowledge of the internal code.

7.1.1 Test Cases

Table 7.1 System Testing Table

S.No	Test Case	Expected Result	Result	Remarks (If Fails)
1	User Registration	New user account created successfully	Pass	Fails if mandatory fields are empty or database connection is unavailable.
2	User Login	User successfully logs into the system	Pass	Fails if username/password is incorrect or account is not activated.
3	User Authentication	Authorized users gain access	Pass	Fails if session validation or authentication process fails.
4	Dataset Upload	CSV dataset uploaded successfully	Pass	Fails if file format is invalid, corrupted, or exceeds size limits.
5	Dataset Validation	Required columns are validated	Pass	Fails if H_2, CO, C_2H_4, or C_2H_2 columns are missing.
6	Feature Extraction	Features generated successfully	Pass	Fails if dataset contains invalid or insufficient data.
7	Fault Prediction	Transformer fault is correctly classified	Pass	Fails if ML model is not loaded or prediction process encounters an error.
8	RUL Prediction	Remaining Useful Life is predicted	Pass	Fails if prediction model is unavailable or input data is invalid.
9	Health Score Calculation	Health score is generated correctly	Pass	Fails if RUL prediction is unavailable or calculation error occurs.
10	SHAP Analysis	Important features are displayed	Pass	Fails if SHAP report is missing or explanation data cannot be generated.
11	Dashboard Visualization	Charts and analytics are displayed	Pass	Fails if report data is unavailable or visualization scripts fail to load.
12	Sample Dataset Selection	Selected sample file loads successfully	Pass	Fails if sample dataset file is missing from the server.
13	Prediction Report Generation	Prediction report is generated	Pass	Fails if prediction results are unavailable or report generation fails.
14	Excel Export	Report is downloaded successfully	Pass	Fails if export library or file generation process encounters an error.
15	User Logout	User session is terminated successfully	Pass	Fails if session data is not cleared properly.
16	Admin Login	Administrator gains access to admin panel	Pass	Fails if admin credentials are incorrect.
17	User Activation	User account status	Pass	Fails if user record is not found in the

		changes to activated		database.
18	User Deactivation	User account is deactivated successfully	Pass	Fails if database update operation fails.
19	User Update	User information is updated successfully	Pass	Fails if invalid data is entered or database update fails.
20	User Deletion	Selected user is removed from the system	Pass	Fails if user record does not exist or deletion operation fails.

7.2 Results Discussion

The proposed Predictive Grid Intelligence Framework for Distribution Substation Maintenance Failure Prevention Through Advanced Machine Learning Analytics was evaluated based on its functional requirements and predictive maintenance capabilities. The system focuses on analyzing distribution substation data, monitoring electrical equipment conditions, identifying abnormal operating patterns, and predicting potential equipment failures. The testing process verifies the proper functioning of data collection, data preprocessing, machine learning model training, failure prediction, and result visualization modules.

The input validation tests ensure that missing, incorrect, and invalid equipment parameter values are identified before they are processed by the prediction models. The data preprocessing module prepares the collected substation data by handling inconsistencies, removing unnecessary information, and organizing the required input features. The machine learning prediction module analyzes equipment parameters such as voltage, current, temperature, load conditions, and other relevant operational indicators to identify possible failure conditions.

The machine learning models are evaluated using separate training and testing datasets. The models are trained using historical distribution substation data, while the testing dataset is used to evaluate their ability to predict previously unseen failure conditions. The performance of the models is measured using four evaluation metrics: Accuracy, Precision, Recall, and F1-Score. Accuracy measures the overall correctness of predictions, Precision indicates how many predicted failures are actually failures, Recall measures how many actual failures are correctly identified, and the F1-Score represents the balance between Precision and Recall.

The results demonstrate the potential of advanced machine learning analytics in predicting equipment failures and supporting preventive maintenance activities. The framework helps identify abnormal operating conditions at an early stage, enabling maintenance teams to take corrective

action before major equipment breakdowns occur. It also supports better maintenance planning, reduces unexpected power interruptions, minimizes equipment damage, and improves the reliability and operational efficiency of distribution substations.

Overall, the proposed framework provides an intelligent and data-driven approach to distribution substation maintenance. By integrating machine learning techniques with predictive analytics, the system supports failure prevention, improves equipment monitoring, and contributes to a more reliable and efficient electrical power distribution network.

.

CHAPTER 8

CONCLUSION

This project successfully developed a Machine Learning-based Transformer Fault Detection and Remaining Useful Life (RUL) Prediction System for predictive maintenance applications in power distribution systems. By utilizing Dissolved Gas Analysis (DGA) data, the proposed framework effectively identifies transformer operating conditions and provides valuable insights into equipment health and reliability.

Advanced machine learning algorithms such as LightGBM and XGBoost were employed to perform fault classification and RUL prediction. The system accurately classifies transformer conditions into Normal, Partial Discharge, Thermal Fault, and Electrical Fault categories while estimating the remaining operational life of the transformer. Statistical feature extraction and health score calculation further enhance the monitoring process.

The integration of Explainable Artificial Intelligence (XAI) through SHAP analysis improves model interpretability by highlighting the most influential features responsible for prediction outcomes. Additionally, the Django-based web platform provides a user-friendly environment for dataset management, prediction analysis, visualization, and administrative monitoring.

The developed system enables early fault detection, improves maintenance planning, reduces the risk of unexpected transformer failures, and supports reliable operation of power distribution networks. Overall, the combination of Machine Learning, Explainable AI, and web technologies provides an efficient, scalable, and intelligent solution for transformer condition monitoring and predictive maintenance.

Future enhancements may include real-time sensor integration, IoT-based monitoring, cloud deployment, and the application of deep learning models to further improve prediction accuracy and system performance.

CHAPTER 9

FUTURE ENHANCEMENTS

The proposed system provides a foundation for intelligent distribution substation monitoring and predictive maintenance using advanced machine learning analytics. It can be further enhanced in the following directions:

1. **Real-Time Sensor Integration:** IoT sensors can be integrated for continuous monitoring of transformer temperature, humidity, oil quality, voltage, current, and dissolved gas concentrations to enable real-time fault detection and instant alerts.

2. **SCADA System Integration:** The system can be connected with Supervisory Control and Data Acquisition (SCADA) systems to automatically collect operational data from substations and electrical equipment.

3. **Advanced Deep Learning Models:** Advanced algorithms such as CNN, LSTM, GRU, and Transformer Networks can be implemented to improve fault prediction accuracy and identify complex patterns in equipment operating data.

4. **Cloud-Based Deployment:** The system can be deployed on cloud platforms to support large-scale substation monitoring, centralized data storage, scalability, and remote access.

5. **Mobile Application Development:** Android and iOS applications can be developed to help maintenance engineers monitor equipment health, view failure predictions, and receive instant fault notifications.

6. **Automated Maintenance Recommendation System:** An intelligent recommendation module can be added to suggest suitable maintenance actions based on fault severity, equipment health scores, and Remaining Useful Life (RUL) predictions.

7. **Digital Twin Technology:** Digital twin models can be developed for transformers and other substation equipment to support virtual monitoring, equipment simulation, and predictive analysis.

8. **Multi-Equipment Monitoring:** The system can be extended to monitor additional substation equipment, including circuit breakers, current transformers, voltage transformers, and switchgear.

REFERENCES

[1] V. Vita, “Predictive Maintenance for Distribution System Operators in Increasing Transformers’ Reliability,” 2023.

[2] V. Biradar and D. Kakeri, “Machine Learning Based Predictive Maintenance in Distribution Transformers,” 2024.

[3] M. Khalid, Z. Bin, and A. Khan, “Deep Learning for Transformer Fault Diagnosis Using Dissolved Gas Analysis,” 2023.

[4] R. Sharma and P. K. Jain, “Transformer Health Index Prediction Using Random Forest,” 2023.

[5] J. He et al., “Condition Monitoring of Transformers Using Explainable AI,” 2024.

[6] M. Duval, “A Review of Faults Detectable by Gas-in-Oil Analysis in Transformers,” *IEEE Electrical Insulation Magazine*, vol. 18, no. 3, pp. 8–17, 2002.

[7] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] K. Cho et al., “Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[12] A. Vaswani et al., “Attention Is All You Need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 5998–6008.

[13] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 4765–4774.

[14] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2022.

[15] IEEE, *IEEE Guide for the Interpretation of Gases Generated in Oil-Immersed Transformers*, IEEE Std C57.104-2019, 2019.